



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY  
(AN AUTONOMOUS INSTITUTION)**

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,



Narayanaguda, Hyderabad-500029.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

NAME OF THE LAB: Software Testing Methodologies

**B.Tech IV YEAR I SEM (KR23)**  
**ACADEMIC YEAR 2026-27**



# KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad



## Certificate

This is to certify that following is a Bonafide Record of the workbook task done by

\_\_\_\_\_ bearing Roll No \_\_\_\_\_ of

\_\_\_\_\_ Branch of \_\_\_\_\_ year B.Tech Course in the

\_\_\_\_\_ Subject during the Academic year \_\_\_\_\_ & \_\_\_\_\_  
under our supervision.

Number of experiments completed: \_\_\_\_\_

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner

## Daily Laboratory Assessment Sheet

Name of the Lab:

Name of the Student:

Class:

HT.No:

| S.No. | Name of the Experiment | Date | Observation<br>Marks (3M) | Record<br>Marks<br>(4M) | Viva Voice<br>Marks (3M) | Total Marks<br>(10M) | Signature<br>of Faculty |
|-------|------------------------|------|---------------------------|-------------------------|--------------------------|----------------------|-------------------------|
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       |                        |      |                           |                         |                          |                      |                         |
|       | <b>TOTAL</b>           |      |                           |                         |                          |                      |                         |

## CONTENT

| S.NO | WEEK    | TOPIC                                                                                                                                          | PAGENO |
|------|---------|------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1    | -       | Syllabus                                                                                                                                       |        |
| 2    | -       | Course Objectives, Out comes                                                                                                                   |        |
| 3    | Week 1  | Recording in context sensitive mode and analog mode                                                                                            |        |
| 4    | Week 2  | GUI checkpoint for single property                                                                                                             |        |
| 5    | Week 3  | GUI checkpoint for single object/window                                                                                                        |        |
| 6    | Week 4  | GUI checkpoint for multiple objects<br>(a) Bitmap checkpoint for object/window                                                                 |        |
| 7    | Week 5  | (b) Bitmap checkpoint for screen area                                                                                                          |        |
| 8    | Week 6  | Database checkpoint for Default check                                                                                                          |        |
| 9    | Week 7  | Database checkpoint for custom check                                                                                                           |        |
| 10   | Week 8  | Database checkpoint for runtime record check<br>a) Data driven test for dynamic test data submission<br>b) Data driven test through flat files |        |
| 11   | Week 9  | c) Data driven test through front grids<br>d) Data driven test through excel test                                                              |        |
| 12   | Week10  | e) Batch testing without parameter passing<br>f) Batch testing with parameter passing                                                          |        |
| 13   | Week 11 | Data driven batch                                                                                                                              |        |
| 14   | Week 12 | Silent mode test execution without any interruption                                                                                            |        |
| 15   | Week 13 | Test case for calculator in windows application                                                                                                |        |

# SYLLABUS

## 1. List of Experiments:

1. Recording in context sensitive mode and analog mode
2. GUI checkpoint for single property
3. GUI checkpoint for single object/window
4. GUI checkpoint for multiple objects
  - a) Bitmap checkpoint for object/window
  - b) Bitmap checkpoint for screen area
5. Database checkpoint for Default check
6. Database checkpoint for custom check
7. Database checkpoint for runtime record check
  - a) Data driven test for dynamic test data submission
  - b) Data driven test through flat files
  - c) Data driven test through front grids
  - d) Data driven test through excel test
  - e) Batch testing without parameter passing
  - f) Batch testing with parameter passing
8. Data driven batch
9. Silent mode test execution without any interruption
10. Test case for calculator in windows application

## TEXT BOOKS:

1. software Testing techniques - Baris Beizer, Dreamtech, second edition.
2. Software Testing Tools — Dr. K. V. K. K. Prasad, Dreamtech.

## REFERENCE BOOKS:

1. The craft of software testing - Brian Marick, Pearson Education.
2. Software Testing Techniques — SPD(Oreille)
3. Software Testing in the Real World — Edward Kit, Pearson.
4. Effective methods of Software Testing, Perry, John Wiley.
5. Art of Software Testing — Meyers, John Wiley.

Software to be used: The students must use Selenium.

1. Install Java JDK 11/17 (or higher) and set JAVA\_HOME and Path.
2. Install Eclipse IDE for Java Developers.
3. Create a Maven Project in Eclipse (File > New > Maven Project).
4. In pom.xml, add Selenium dependency (selenium-java 4.x) and save.
5. Right-click project > Maven > Update Project (Force Update).
6. Create package: src/main/java > com.test. selenium.
7. Ensure Google Chrome or Microsoft Edge is installed and updated.

Note: Selenium Manager (Selenium 4.6+) usually downloads the correct driver automatically. So you normally do not need to download chromedriver.exe or set System.setProperty for drivers.

Course Objectives: The course will help to

1. To provide knowledge of Software Testing Methods.
2. To develop skills in software test automation and management using latest tools.
3. To expose the advanced software testing topics, such as object-oriented software testing methods.
4. To develop skills in software test automation and management using latest tools.
5. To discuss various software testing issues and solutions in software unit test, integration and system testing.

Course Outcomes: After learning the concepts of this course, the student is able to

1. Design and develop the best test strategies in accordance to the development model.
2. Outline the necessity of testing, debugging using program control flow.
3. Apply transaction flow, data flow testing to unit and integration testing. Able to test a domain or an application and identify the nice and ugly domains
4. Understand the testing of state graphs.
5. Analyze graph matrices for optimizing the code and use of testing tools like WinRunner, JMeter.



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY**  
(AN AUTONOMOUS INSTITUTION)  
**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,**  
**Narayanguda, Hyderabad – 500029**



## **Department of Computer Science and Engineering**

### **Vision of the Institution:**

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

### **Mission of the Institution:**

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY**  
(AN AUTONOMOUS INSTITUTION)  
**Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,**  
**Narayanguda, Hyderabad – 500029**



## **Department of Computer Science and Engineering**

### **Vision of the Department:**

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

### **Mission of the Department:**

- To provide faculty with state-of-the-art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY**  
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,  
Narayanguda, Hyderabad – 500029



**Department of Computer Science and Engineering**

**PROGRAM OUTCOMES (POs):**

**PO1: Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

**PO2: Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

**PO3: Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

**PO4: Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

**PO5: Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

**PO6: The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

**PO7: Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

**PO8: Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

**PO9: Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

**PO10: Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend

and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

**PO11: Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

**PO12: Life-long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



**KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY**  
(AN AUTONOMOUS INSTITUTION)  
Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,  
Narayanguda, Hyderabad – 500029



## **Department of Computer Science and Engineering**

### **PROGRAM SPECIFIC OUTCOMES (PSOs)**

**PSO1:** An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftments.

**PSO2:** Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.



## **Department of Computer Science and Engineering**

### **PROGRAM EDUCATIONAL OBJECTIVES (PEOs)**

**PEO1:** Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

**PEO2:** Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

**PEO3:** Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

**PEO4:** Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.

## **PROGRAMS**

### **Week1.**

#### **Experiment 1: Recording in Context Sensitive Mode and Analog Mode (Selenium IDE)**

Tools/Requirements:

- Selenium IDE extension (Chrome/Edge)
- Any demo site for drag-and-drop (optional)

Procedure:

1. Open Chrome/Edge and install the Selenium IDE extension from the browser extensions store.
2. Click the Extensions icon and open Selenium IDE.
3. Click 'Create a new project' and enter a project name.
4. Click 'Add new test' and name the test case (e.g., TC\_Record\_Google).
5. Click the red 'Record' button.
6. Enter the base URL (e.g., <https://www.google.com>) and click 'Start Recording'.
7. Perform actions: click the search box, type a keyword, press Enter, click a result, then go back.
8. Stop recording and review the generated commands (open, click, type, assert).
9. Run the test using the 'Run current test' (play) button.
10. For analog-style actions: try recording a drag-and-drop or slider movement on a demo site and replay it.
11. Save the project in Selenium IDE.

Notes:

- Selenium IDE is mainly context-sensitive (element based). True analog recording is limited.
- For complex gestures use Selenium WebDriver code with the Actions class.

#### **Output:**

```
Running 'TC_Record_Google'  
open | https://www.google.com |  
type | name=q | selenium  
assertTitle | Google |  
Test completed successfully
```

Result: PASS

## Week2

### Experiment 2: GUI Checkpoint for Single Property (Title/Text/Attribute)

Sample Java Code:

// Exp 2: Single Property Checkpoint (Page Title)

```
import javax.swing.JFrame;

public class Exp2_SinglePropertyCheckpoint {
    public static void main(String[] args) throws InterruptedException {
        JFrame frame = new JFrame("Google");
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null); frame.setVisible(true);

        String expectedTitle = "Google";
        String actualTitle = frame.getTitle();

        System.out.println("Actual Title : " + actualTitle);
        System.out.println("Expected Title : " + expectedTitle);

        if (actualTitle.equals(expectedTitle)) {
            System.out.println("PASS: Title is correct");
        } else {
            System.out.println("FAIL: Title is incorrect");
        }

        Thread.sleep(2000);
        frame.dispose();
    }
}
```

#### Output

```
Actual Title : Google
Expected Title : Google
PASS: Title is correct
```

## Week3

### Experiment 3: GUI Checkpoint for Single Object/Window (Displayed/Enabled)

#### Sample Java Code:

// Exp 3: Single Object Checkpoint (Element displayed and enabled) import javax.swing.\*;

```
public class Exp3_SingleObjectCheckpoint {
    public static void main(String[] args) throws InterruptedException
    {
        JFrame frame = new JFrame("Single Object Checkpoint");
        JTextField searchBox = new JTextField(20);

        JPanel panel = new JPanel();
        panel.add(new JLabel("Search:"));
        panel.add(searchBox);
        frame.add(panel);
        frame.setSize(400, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        Thread.sleep(500);

        boolean displayed = searchBox.isShowing();
        boolean enabled = searchBox.isEnabled();

        System.out.println("Displayed: " + displayed);
        System.out.println("Enabled : " + enabled);

        System.out.println(displayed && enabled
            ? "PASS: Search box is ready"
            : "FAIL: Search box is not ready");

        Thread.sleep(2000); frame.dispose();
    }
}
```

#### Output

Displayed: true

Enabled : true

PASS: Search box is ready

## Week4

### Experiment 4(a): Bitmap Checkpoint for Object/Window (Screenshot Comparison)

```
import javax.swing.*;
import javax.imageio.ImageIO;
import java.awt.*;
import java.awt.image.BufferedImage; import java.io.File;
public class NewTest {
    public static void main(String[] args) throws Exception {
        JFrame frame = new JFrame("Bitmap Checkpoint Test");
        JTextField searchBox = new JTextField("Search here", 20);
        JPanel panel = new JPanel();
        panel.add(searchBox);
        frame.add(panel);
        frame.setSize(350, 120);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null); frame.setVisible(true);
        Thread.sleep(1000);
        File folder = new File("Screenshots");
        if (!folder.exists()) folder.mkdir();
        BufferedImage currentImage = new BufferedImage( searchBox.getWidth(), searchBox.getHeight(),
        BufferedImage.TYPE_INT_RGB);
        Graphics2D g2 = currentImage.createGraphics();
        searchBox.paint(g2);
        g2.dispose();
        File currentFile = new File("Screenshots/current_searchbox.png");
        ImageIO.write(currentImage, "png", currentFile);
        System.out.println("Current screenshot saved:");
        System.out.println(currentFile.getAbsolutePath());

        File baselineFile = new File("Screenshots/baseline_searchbox.png");
        if (!baselineFile.exists()) {
            System.out.println("Baseline image not found.");
            System.out.println("Rename current_searchbox.png as baseline_searchbox.png");
        } else {
            boolean result = compareImages(baselineFile, currentFile);
            if (result) {
                System.out.println("BITMAP CHECKPOINT : TEST PASS");
                System.out.println("Images are identical.");
            } else {
                System.out.println("BITMAP CHECKPOINT : TEST FAIL");
                System.out.println("Images are different.");
            }
        }
        Thread.sleep(2000);
        frame.dispose();
    }

    public static boolean compareImages(File img1, File img2) throws Exception {
```

```

BufferedImage image1 = ImageIO.read(img1);
BufferedImage image2 = ImageIO.read(img2);
if (image1.getWidth() != image2.getWidth() || image1.getHeight() != image2.getHeight())
    return false;

for (int x = 0; x < image1.getWidth(); x++) {
    for (int y = 0; y < image1.getHeight(); y++) {
        if (image1.getRGB(x, y) != image2.getRGB(x, y))
            return false;
    }
}
return true;
}
}

```

### Output

Current screenshot saved:

.../Screenshots/current\_searchbox.png

BITMAP CHECKPOINT : TEST

PASS

Images are identical.



## Week5

### Experiment 4(b): Bitmap Checkpoint for Screen Area (Crop + Compare)

```
import javax.swing.*;
import javax.imageio.ImageIO;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;

public class NewTest {
    public static void main(String[] args) throws Exception {
        JFrame frame = new JFrame("Bitmap Area Checkpoint");
        JPanel panel = new JPanel(new FlowLayout());
        panel.add(new JLabel("Google Search"));
        panel.add(new JTextField(25));
        panel.add(new JButton("Search"));
        frame.add(panel);
        frame.setSize(800, 600);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        Thread.sleep(1000);

        BufferedImage fullImage = new BufferedImage(frame.getWidth(), frame.getHeight(),
        BufferedImage.TYPE_INT_RGB);

        Graphics2D g2 = fullImage.createGraphics();
        frame.paint(g2);
        g2.dispose();

        File folder = new File("SeleniumScreenshots");

        if (!folder.exists())

            folder.mkdirs();

        int width = Math.min(800, fullImage.getWidth());
        int height = Math.min(600, fullImage.getHeight());

        BufferedImage currentArea = fullImage.getSubimage(0, 0, width, height);

        File currentFile = new File("SeleniumScreenshots/current_google_area.png");
        ImageIO.write(currentArea, "png", currentFile);
        System.out.println("Current screenshot saved:");
        System.out.println(currentFile.getAbsolutePath());

        File baselineFile = new File("SeleniumScreenshots/baseline_google_area.png");

        if (!baselineFile.exists()) {
            System.out.println("Baseline image not found.");
            System.out.println("Rename current_google_area.png to baseline_google_area.png");
        }
    }
}
```

```

    } else {
        BufferedImage baseline = ImageIO.read(baselineFile);
        double similarity = compareImages(baseline, currentArea);
        System.out.printf("Image Similarity : %.2f%%\n", similarity);
        if (similarity >= 95)
            System.out.println("BITMAP CHECKPOINT : TEST PASS");
        else
            System.out.println("BITMAP CHECKPOINT : TEST FAIL");
    }
    Thread.sleep(2000);
    frame.dispose();
}

public static double compareImages(BufferedImage img1, BufferedImage img2) {
    if (img1.getWidth() != img2.getWidth() ||
        img1.getHeight() != img2.getHeight()) return 0;

    long totalPixels = (long) img1.getWidth() * img1.getHeight();
    long samePixels = 0;
    for (int x = 0; x < img1.getWidth(); x++) {
        for (int y = 0; y < img1.getHeight(); y++) {
            if (img1.getRGB(x, y) == img2.getRGB(x, y)) samePixels++;
        }
    }
    return ((double) samePixels / totalPixels) * 100;
}

```

#### Output

Current screenshot saved:  
 ../SeleniumScreenshots/current\_google\_area.  
 png Image Similarity : 100.00%  
 BITMAP CHECKPOINT : TEST PASS

Google Search

Search

## Week6

### Experiment 5: Database Checkpoint – Default Check (Record Exists)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class NewTest {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection( "jdbc:h2:./StudentDB", "sa", "");
            Statement stmt = con.createStatement();
            stmt.execute("CREATE TABLE IF NOT EXISTS STUDENT (" + "ID INT PRIMARY KEY, " +
                "NAME VARCHAR(50), " + "EMAIL VARCHAR(100))");
            stmt.execute("DELETE FROM STUDENT");
            stmt.executeUpdate("INSERT INTO STUDENT VALUES " +
                "(1, 'Ajay', 'ajay@gmail.com')");
            ResultSet rs = stmt.executeQuery(
                "SELECT * FROM STUDENT WHERE ID = 1");
            if (rs.next()) {
                System.out.println("Record Exists");
                System.out.println("ID : " + rs.getInt("ID"));
                System.out.println("Name : " + rs.getString("NAME"));
                System.out.println("Email : " + rs.getString("EMAIL"));
                System.out.println("TEST PASSED");
            } else {
                System.out.println("Record Not Found");
                System.out.println("TEST FAILED");
            }

            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

#### Output

```
Record Exists ID: 1
Name : Ajay
Email : ajay@gmail.com
TEST PASSED
```

## Week7

### Experiment 6: Database Checkpoint – Custom Check (Validate Specific Value)

```
import java.sql.*;
public class DatabaseCheckpointCustomCheck {
    static final String DB_URL = "jdbc:sqlite:student.db";
    public static void setupDatabase() throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
            Statement stmt = conn.createStatement()) {
            stmt.execute("CREATE TABLE IF NOT EXISTS students (" + "id INTEGER PRIMARY KEY, name
TEXT, marks INTEGER)");
            stmt.execute("DELETE FROM students");
            stmt.execute("INSERT INTO students VALUES (1, 'Amit', 78)");
            stmt.execute("INSERT INTO students VALUES (2, 'Neha', 45)");
            stmt.execute("INSERT INTO students VALUES (3, 'Ravi', 92)");
        }
    }

    public static boolean customCheckpoint(int studentId, String expectedStatus)
throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
            PreparedStatement ps = conn.prepareStatement("SELECT marks FROM students WHERE id = ?")) {
            ps.setInt(1, studentId);
            ResultSet rs = ps.executeQuery();
            if (!rs.next()) {
                System.out.println("FAIL: No record found for id " + studentId);
                return false;
            }
            int marks = rs.getInt("marks");
            String actualStatus = (marks >= 50) ? "Pass" : "Fail";
            if (actualStatus.equals(expectedStatus)) {
                System.out.println("PASS: Student " + studentId + " status = " + actualStatus + " (Custom check
matched)");
                return true;
            } else {
                System.out.println("FAIL: Student " + studentId + " expected " + expectedStatus + ", got " +
actualStatus);
                return false;
            }
        }
    }

    public static void main(String[] args) throws Exception { setupDatabase();
        customCheckpoint(1, "Pass");
        customCheckpoint(2, "Fail");
        customCheckpoint(3, "Pass");
    }
}
```

**Compile / Run Commands**

```
javac -cp sqlite-jdbc-3.34.0.jar DatabaseCheckpointCustomCheck.java  
java -cp ".;sqlite-jdbc-3.34.0.jar" DatabaseCheckpointCustomCheck
```

**Output**

PASS: Student 1 status = Pass (Custom check matched)

PASS: Student 2 status = Fail (Custom check matched)

PASS: Student 3 status = Pass (Custom check matched)

## Week8

### Experiment 7(a): Data Driven Test – Dynamic Test Data Submission

```
import java.util.Random;

public class DynamicDataTest {

public static double calculateDiscount(double price, String category) { switch (category) {
case "electronics": return price * 0.9; case "clothing": return price * 0.8; default: return price;
}
}

public static void main(String[] args) {
String[] categories = {"electronics", "clothing", "grocery"}; Random rand = new Random();
System.out.println("---- Dynamic Test Data Submission      ");

for (int i = 1; i <= 5; i++) {
int price = 100 + rand.nextInt(901);
String category = categories[rand.nextInt(categories.length)];
double result = calculateDiscount(price, category);
System.out.printf(
"Test %d: price=%d, category=%s -> discounted=%.2f%n", i, price, category, result);
}
}
}
```

### Compile / Run Commands

```
javac DynamicDataTest.java
java DynamicDataTest
```

### Output

```
---- Dynamic Test Data Submission ----
Test 1: price=527, category=electronics -> discounted=474.30
Test 2: price=304, category=clothing -> discounted=243.20
Test 3: price=812, category=grocery -> discounted=812.00
Test 4: price=450, category=electronics -> discounted=405.00
Test 5: price=265, category=clothing -> discounted=212.00
```

### Experiment 7(b): Data Driven Test – Through Flat Files (CSV/TXT)

```
import java.io.*;

public class FlatFileTest {
    public static double calculateDiscount(double price, String category) { switch (category) {
    case "electronics": return price * 0.9; case "clothing": return price * 0.8; default: return price;
    }
    }
    public static void createFlatFile() throws IOException {
    try (PrintWriter pw = new PrintWriter(new FileWriter("testdata.txt")))
    {
    pw.println("500,electronics");
    pw.println("300,clothing"); pw.println("150,grocery");
    }
    }
    public static void flatFileTest() throws IOException {
    System.out.println("---- Data Driven Test via Flat File  ");
    try (BufferedReader br = new BufferedReader( new FileReader("testdata.txt")))
    {
    String line;
    while ((line = br.readLine()) != null) { String[] parts = line.split(",");
    double price = Double.parseDouble(parts[0]); String category = parts[1];
    double result = calculateDiscount(price, category);
    System.out.printf(
        "price=%.1f, category=%s -> discounted=%.2f%n", price, category, result);
    }
    }
    }

    public static void main(String[] args) throws IOException { createFlatFile();
    flatFileTest();
    }
}
```

#### Compile / Run Commands

```
javac FlatFileTest.java
```

```
java FlatFileTest
```

Output

```
---- Data Driven Test via Flat File ----
```

```
price=500.0, category=electronics -> discounted=450.00
```

```
price=300.0, category=clothing -> discounted=240.00
```

```
price=150.0, category=grocery -> discounted=150.0
```

## Week9

### Experiment 7(c): Data Driven Test – Through Front Grids (Web Tables)

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
public class FrontEndGridTest {
    public static double calculateDiscount(double price, String category) { switch (category) {
    case "electronics": return price * 0.9; case "clothing": return price * 0.8; default: return price;
    }
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame("Front End Grid - Data Driven Test");
        String[] columns = {"Price", "Category", "Result"};
        DefaultTableModel model = new DefaultTableModel(columns, 0);

        Object[][] gridData = {
            {500.0, "electronics"},
            {300.0, "clothing"},
            {150.0, "grocery"}
        };

        for (Object[] row : gridData) { double price = (double) row[0];
        String category = (String) row[1];
        double result = calculateDiscount(price, category);
        model.addRow(new Object[]{
            price, category, String.format("%.2f", result)});
        }
        JTable table = new JTable(model);
        frame.add(new JScrollPane(table));
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

**Compile / Run Commands**

```
javac FrontEndGridTest.java  
java FrontEndGridTest
```

**Output**

| Price | Category    | Result |
|-------|-------------|--------|
| 500.0 | electronics | 450.00 |
| 300.0 | clothing    | 240.00 |
| 150.0 | grocery     | 150.00 |

### Experiment 7(d): Data Driven Test – Through Excel

```
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook; import java.io.*;
public class ExcelDrivenTest {
    public static double calculateDiscount(double price, String category) { switch (category) {
        case "electronics": return price * 0.9; case "clothing": return price * 0.8; default: return price;
    }
}
public static void createExcelFile() throws IOException {
    Workbook wb = new XSSFWorkbook();
    Sheet sheet = wb.createSheet("TestData");

    Object[][] data = {
        {"Price", "Category"},
        {500.0, "electronics"},
        {300.0, "clothing"},
        {150.0, "grocery"}
    };
    for (int r = 0; r < data.length; r++) { Row row = sheet.createRow(r);
    for (int c = 0; c < data[r].length; c++) { Cell cell = row.createCell(c);
    if (data[r][c] instanceof Double) cell.setCellValue((Double) data[r][c]);
    else
        cell.setCellValue((String) data[r][c]);
    }
    }
    try (FileOutputStream fos = new FileOutputStream("testdata.xlsx")) { wb.write(fos);
    }
    wb.close();
}

public static void excelDrivenTest() throws IOException {
    System.out.println("---- Data Driven Test via Excel ");
    try (FileInputStream fis = new FileInputStream("testdata.xlsx");
        Workbook wb = new XSSFWorkbook(fis)) {
        Sheet sheet = wb.getSheetAt(0);
        for (Row row : sheet) {
            if (row.getRowNum() == 0) continue;
            double price = row.getCell(0).getNumericCellValue();
            String category = row.getCell(1).getStringCellValue();
            double result = calculateDiscount(price, category);
            System.out.printf("price=%.1f, category=%s -> discounted=%.2f%n", price, category, result);
        }
    }
}

public static void main(String[] args) throws IOException { createExcelFile();
    excelDrivenTest();
}
```

### **Compile / Run Commands**

```
javac -cp ".;poi-5.2.5.jar;poi-ooxml-5.2.5.jar;poi-ooxml-lite-5.2.5.jar;xmlbeans-5.2.0.jar;commons-collections4-4.4.jar;commons-compress-1.26.0.jar;commons-io-2.15.1.jar;log4j-api-2.20.0.jar" ExcelDrivenTest.java
```

```
java -cp ".;poi-5.2.5.jar;poi-ooxml-5.2.5.jar;poi-ooxml-lite-5.2.5.jar;xmlbeans-5.2.0.jar;commons-collections4-4.4.jar;commons-compress-1.26.0.jar;commons-io-2.15.1.jar;log4j-api-2.20.0.jar" ExcelDrivenTest
```

### **Output**

---- Data Driven Test via Excel ----

price=500.0, category=electronics -> discounted=450.00

price=300.0, category=clothing -> discounted=240.00

price=150.0, category=grocery -> discounted=150.00

## Week10

### Experiment 7(e): Batch Testing – Without Parameter Passing (Suite Run)

```
public class BatchNoParams {  
    public static void testAddition() {  
        assert 2 + 3 == 5;  
        System.out.println("test_addition: PASS");  
    }  
    public static void testSubtraction() { assert 5 - 2 == 3;  
        System.out.println("test_subtraction: PASS");  
    }  
    public static void main(String[] args) {  
        System.out.println("---- Batch Testing (No Parameters)  ");  
        testAddition(); testSubtraction();  
    }  
}
```

### Compile / Run Commands

```
javac BatchNoParams.java  
java -ea BatchNoParams
```

### Output

```
---- Batch Testing (No Parameters) ----  
test_addition: PASS  
test_subtraction: PASS
```

### Experiment 7(f): Batch Testing – With Parameter Passing (Suite Parameters)

```
public class BatchWithParams {
    static void testOperation(int a, int b, char op, int expected) {
        int result = 0;
        switch (op) {
            case '+': result = a + b;
                     break;
            case '-': result = a - b;
                     break;
            case '*': result = a * b;
                     break;
        }
        String status = (result == expected) ? "PASS" : "FAIL";
        System.out.printf("Test: %d %c %d = %d (expected %d) -> %s%n", a, op, b, result, expected, status);
    }
    public static void main(String[] args) {
        int[][] testCases = {{2, 3, '+', 5},{10, 4, '-', 6},{3, 3, '*', 9},{5, 5, '-', 1}};
        System.out.println("---- Batch Testing (With Parameters)      ");
        for (int[] tc : testCases) {
            testOperation(tc[0], tc[1], (char) tc[2], tc[3]);
        }
    }
}
```

#### Compile / Run Commands

```
javac BatchWithParams.java
java BatchWithParams
```

#### Output

```
---- Batch Testing (With Parameters) ----
Test: 2 + 3 = 5 (expected 5) -> PASS
Test: 10 - 4 = 6 (expected 6) -> PASS
Test: 3 * 3 = 9 (expected 9) -> PASS
Test: 5 - 5 = 0 (expected 1) -> FAIL
```

## Week 11

### Experiment 8: Data Driven Batch (Batch + Multiple Data Sets)

```
public class DataDrivenBatch {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createCsvFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("batch_data.csv"))) {
            pw.println("Price,Category,ExpectedResult");
            pw.println("500,electronics,450");
            pw.println("300,clothing,240");
            pw.println("150,grocery,150");
            pw.println("1000,electronics,900");
        }
    }

    public static void dataDrivenBatch() throws IOException {
        System.out.println("---- Data Driven Batch Test");
        int passCount = 0, failCount = 0;
        try (BufferedReader br = new BufferedReader(new FileReader("batch_data.csv"))) {
            String line = br.readLine();
            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                double price = Double.parseDouble(parts[0]);
                String category = parts[1];
                double expected = Double.parseDouble(parts[2]);
                double actual = calculateDiscount(price, category);
                if (actual == expected) {
                    System.out.printf("PASS: %s, price=%.1f -> %.1f%n", category, price, actual);
                    passCount++;
                } else {
                    System.out.printf("FAIL: %s, price=%.1f -> got %.1f, expected %.1f%n", category, price, actual, expected);
                    failCount++;
                }
            }
        }
        System.out.println("\nBatch Summary: " + passCount + " Passed, " + failCount + " Failed");
    }

    public static void main(String[] args) throws IOException {
        createCsvFile();
        dataDrivenBatch();
    }
}
```

**Compile / Run Commands**

```
javac DataDrivenBatch.java  
java DataDrivenBatch
```

**Output**

```
---- Data Driven Batch Test ----  
PASS: electronics, price=500.0 -> 450.0  
PASS: clothing, price=300.0 -> 240.0  
PASS: grocery, price=150.0 -> 150.0  
PASS: electronics, price=1000.0 -> 900.0
```

Batch Summary: 4 Passed, 0 Failed

## Week 12

### Experiment 9: Silent Mode Test Execution (Headless)

```
import java.util.logging.*;

import java.io.IOException;

public class SilentModeTest {

    static Logger logger = Logger.getLogger("SilentTestLogger");

    static void setupLogger() throws IOException {

        logger.setUseParentHandlers(false);

        FileHandler fh = new FileHandler("silent_test_log.txt");

        fh.setFormatter(new SimpleFormatter());

        logger.addHandler(fh);

    }

    static void testCase(String name, Object actual, Object expected) {

        try {

            if (actual.equals(expected)) {

                logger.info(name + ": PASS (actual=" + actual + ", expected=" + expected + ")");

            } else {

                logger.severe(name + ": FAIL (actual=" + actual + ", expected=" + expected + ")");

            }

        } catch (Exception e) {

            logger.severe(name + ": ERROR - " + e.getMessage());

        }

    }

    public static void main(String[] args) throws IOException { setupLogger();
    testCase("Addition Test", 2 + 2, 4);
    testCase("Subtraction Test", 10 - 5, 5);
```

```
testCase("Multiplication Test", 3 * 3, 10);

testCase("String Test", "abc".toUpperCase(), "ABC");

System.out.println("Batch execution completed silently. " + "Check silent_test_log.txt for results.");

}

}
```

### **Compile / Run Commands**

```
javac SilentModeTest.java
java SilentModeTest
type silent_test_log.txt
```

### **Output**

#### **Terminal:**

Batch execution completed silently. Check silent\_test\_log.txt for results.

#### **Log file (message portion):**

```
INFO: Addition Test: PASS (actual=4, expected=4)
INFO: Subtraction Test: PASS (actual=5, expected=5)
SEVERE: Multiplication Test: FAIL (actual=9, expected=10)
INFO: String Test: PASS (actual=ABC, expected=ABC)
```

## Week 13

### Experiment 10 - Test Case for Calculator in Windows Application

```
import java.awt.*;
import java.awt.event.KeyEvent;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;

public class CalculatorTest {
    public static void main(String[] args) throws Exception {
        Runtime.getRuntime().exec("calc.exe");
        Thread.sleep(2000);
        Robot robot = new Robot();
        robot.setAutoDelay(200);

        typeKey(robot, KeyEvent.VK_7);
        typeKey(robot, KeyEvent.VK_ADD);
        typeKey(robot, KeyEvent.VK_5);
        typeKey(robot, KeyEvent.VK_EQUALS);
        Thread.sleep(1000);
        Rectangle screenRect = new Rectangle( Toolkit.getDefaultToolkit().getScreenSize());
        BufferedImage capture = robot.createScreenCapture(screenRect);

        ImageIO.write(capture, "png",new File("calculator_result.png"));
        System.out.println("Test executed: 7 + 5 =");
        System.out.println("Screenshot saved as calculator_result.png -- " + "compare visually to confirm result is 12");
    }

    static void typeKey(Robot robot, int keyCode) {
        robot.keyPress(keyCode);
        robot.keyRelease(keyCode);
    }
}
```

### Compile / Run Commands

```
javac CalculatorTest.java  
java CalculatorTest
```

### Output

Test executed:  $7 + 5 = 12$

Screenshot saved as calculator\_result.png -- compare visually to confirm result is 12

